DAY 08 | HANDS-ON ACTIVITIES

FastAPI Advanced and Security: Problem Statements

Seven problems building a secure Notes API. mongomock in Colab, a real MongoDB in VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Shared Pagination

Level: Warm-up | **Concepts:** Depends(), reusable dependency

Write a pagination dependency (skip >= 0, limit 1 to 50) and use it on two endpoints. Show that invalid values return 422.

Sample run / expected result

```
GET /items?skip=0&limit=5 -> {"skip": 0, "limit": 5}
GET /items?limit=500 -> 422
```

Hints

- Use Query() inside the dependency for rules.

Problem 2: Timing and Request Id

Level: Easy | **Concepts:** middleware, headers

Add middleware that adds X-Process-Time-ms and X-Request-ID (a uuid4) to every response, and prints method, path and status.

Sample run / expected result

```
GET /health 200 1.3 ms
X-Request-ID: 3f2a...
```

Hints

- import uuid; str(uuid.uuid4())

Problem 3: Hash and Verify

Level: Easy | **Concepts:** hashlib.pbkdf2_hmac, salt

Write hash_password(pw) returning salt and hash, and verify_password(pw, stored). Show that the same password gives different hashes for two users.

Sample run / expected result

```
verify_password("secret1", stored) -> True
verify_password("wrong", stored) -> False
```

Hints

- os.urandom(16) for the salt
- hmac.compare_digest to compare

Problem 4: Issue and Read a JWT

Level: Medium | **Concepts:** PyJWT, exp claim

Create a token with sub, role and a 1 minute expiry. Decode it. Then create one that expired in the past and catch ExpiredSignatureError.

Sample run / expected result

```
{'sub': 'hana', 'role': 'member', 'exp': ...}
Expired token rejected
```

Hints

- jwt.encode(payload, SECRET, algorithm="HS256")

Problem 5: Register and Login

Level: Medium | **Concepts:** users collection, unique email, /token

Add POST /register (409 if the username exists) and POST /token that returns an access token for valid credentials, 401 otherwise.

Sample run / expected result

```
POST /token wrong password -> 401
POST /token ok -> {"access_token": "eyJ...", "token_type": "bearer"}
```

Hints

- OAuth2PasswordRequestForm reads username and password from a form.

Problem 6: Protected Notes

Level: Challenging | **Concepts:** current_user dependency, PyMongo

Add GET/POST/DELETE /notes stored in MongoDB. Each note stores its owner; users see and delete only their own notes. Missing token: 401.

Sample run / expected result

```
Hana sees 2 notes, Ravi sees 1
Ravi deleting Hana's note -> 404
```

Hints

- Filter every query by {"owner": user}.

Problem 7: Admin Role

Level: Stretch | **Concepts:** authorisation, 403, dependency factory

Add a role to users. Add GET /admin/notes that lists all notes but only for admins (403 for members). Write tests using dependency_overrides.

Sample run / expected result

```
member -> 403
admin -> all 3 notes
```

Hints

- A function that returns a dependency: require_role("admin").

Ganbatte (頑張って): do your best